Подготовка к работе над проектом

Прежде чем приступить к выполнению проекта, убедитесь, что у вас хватает времени, чтобы погрузиться в эту работу. Внимательно прочитайте задание и выполните все требования.

Что нужно сделать

- Внимательно прочитайте задание и убедитесь, что вы всё поняли.
- Если при изучении задания поймёте, что чего-то не помните, найдите нужный урок и повторите его.
- Не торопитесь, выполняйте все рекомендации в задании поэтапно.
- Если у вас что-то не получается, не сдавайтесь. В описании проекта есть раздел «Подсказки»: там даны более подробные рекомендации по выполнению шагов.
- После окончания проекта отправьте его на автоматическую проверку и посмотрите, все ли требования вы учли. Система подсветит, что можно изменить.
- Как пройдёте автотесты, изучите чек-лист в отдельном уроке. Попробуйте оценить и улучшить проект самостоятельно по нашим рекомендациям.

Как вам урок?

К следующему уроку

CRUD для Yatube

Предварительная настройка

Вам стал доступен репозиторий `api_yatube`, клонируйте его в свою рабочую директорию, например, в *Dev*.

В репозитории — знакомый вам **Yatube**, но не самой последней версии и не весь, а только его бэкенд: приложения, модели. Мы исключили из проекта фронтенд и view-функции: они сейчас не понадобятся.

Создайте и активируйте виртуальное окружение, установите все необходимые пакеты из *requirements.txt*.

Задача

В проекте `api_yatube` есть приложение **posts** с описанием моделей **Yatube**. Вам нужно реализовать API для всех моделей приложения.

Обычно всю логику API выносят в отдельное приложение: при иной организации кода работать в большом проекте со множеством приложений будет неудобно.

Добавьте в проект новое приложение с именем "api" и реализуйте всю логику именно там.

API должен быть доступен только аутентифицированным пользователям. Используйте в проекте аутентификацию по токену **TokenAuthentication**.

Аутентифицированный пользователь авторизован на изменение и удаление своего контента; в остальных случаях доступ предоставляется только для чтения. При попытке изменить чужие данные должен возвращаться код ответа **403 Forbidden**.

Для взаимодействия с ресурсами опишите и настройте такие эндпоинты:

- `api/v1/api-token-auth/` (POST): передаём логин и пароль, получаем токен.

- `api/v1/posts/` (GET, POST): получаем список всех постов или создаём новый пост.
- `api/v1/posts/{post_id}/` (GET, PUT, PATCH, DELETE): получаем, редактируем или удаляем пост с идентификатором`{post_id}`.
- `api/v1/groups/` (GET): получаем список всех групп.
- `api/v1/groups/{group_id}/` (GET): получаем информацию о группе с идентификатором `{group_id}`.
- `api/v1/posts/{post_id}/comments/` (GET): получаем список всех комментариев поста с идентификатором `post_id` (POST): создаём новый комментарий для поста с идентификатором `{post_id}`.
- `api/v1/posts/{post_id}/comments/{comment_id}/` (GET, PUT, PATCH, DELETE): получаем, редактируем или удаляем комментарий с идентификатором `{comment_id}` в посте с `id=post_id`.

В ответ на запросы POST, PUT и PATCH ваш API должен возвращать объект, который был добавлен или изменён.

Обязательное условие: работайте с моделью **Post** через `ModelViewSet`.

Если вы решите, что вьюсеты подойдут и для работы с остальными моделями — не стесняйтесь, применяйте их везде, где можно.

Примеры запросов

Пример POST-запроса с токеном Антона Чехова: добавление нового поста.

POST .../api/v1/posts/

Скопировать код
JSON

```
{
    "text": "Вечером собрались в редакции «Русской мысли», чтобы поговорить о народном театре. Проект Шехтеля всем нравится.",
    "group": 1
}
```

Пример ответа:

Скопировать код
JSON

```
{
    "id": 14,
    "text": "Вечером собрались в редакции «Русской мысли», чтобы поговорить о
народном театре. Проект Шехтеля всем нравится.",
    "author": "anton",
    "image": null,
    "group": 1,
    "pub_date": "2021-06-01T08:47:11.084589Z"
}
```


Пример POST-запроса с токеном Антона Чехова: отправляем новый комментарий к посту с `id=14`.

POST .../api/v1/posts/14/comments/

Скопировать код
JSON


```
{
    "text": "тест тест"
}
```


Пример ответа:

Скопировать код
JSON


```
{
    "id": 4,
    "author": "anton",
    "post": 14,
    "text": "тест тест",
    "created": "2021-06-01T10:14:51.388932Z"
}
```


Пример GET-запроса с токеном Антона Чехова: получаем информацию о группе.

GET *.../api/v1/groups/2/*

Пример ответа:

Скопировать код
JSON


```
{
    "id": 2,
    "title": "Математика",
    "slug": "math",
```

```
    "description": "Посты на тему математики"
}
```

Подсказки

Начните с описания файла *urls.py* на уровне проекта. Он должен включать (*include*) маршруты из приложения *api*.

Если решите везде использовать вьюсеты, то вам понадобится:

- использование регулярных выражений для регистрации некоторых роутеров — это отличный повод повторить урок и заглянуть в шпаргалку;
- разобраться с использованием метода `get_queryset()` вьюсета при передаче дополнительных переменных в запросе;
- при запросе на изменение или удаление данных осуществлять проверку прав. Эту задачу можно реализовать, переопределив методы `perform_update` и `perform_destroy`, например, вот так:

  Скопировать код

  PYTHON

  ```
  def perform_update(self, serializer):
    if serializer.instance.author != self.request.user:
        raise PermissionDenied('Изменение чужого контента запрещено!')
    super(PostViewSet, self).perform_update(serializer)
  ```

  Но можно использовать и иные способы.

Запросы через Postman

Проверять и отлаживать работу API удобно через Postman. Эта программа умеет отправлять запросы, анализировать ответы и сохранять запросы для повторного применения в будущем.

В Postman можно импортировать готовую библиотеку запросов. Тогда при тестировании не придётся писать каждый запрос вручную, а можно будет выбрать нужный запрос из списка и отправить его в один клик.

Можно отправить сразу все запросы из коллекции: они будут отправлены последовательно и в нужной очерёдности, например — сперва запрос на

получение токена, потом — на публикацию поста, и токен, полученный в первом запросе, будет применён во втором.

Можно отправлять запросы по отдельности, но при этом стоит помнить, что для некоторых запросов могут потребоваться данные, предварительно полученные из других запросов.

В директории *postman_collection* сохранена коллекция запросов для отладки и проверки работы текущей версии API Yatube.

Когда проект будет готов обрабатывать запросы к API — импортируйте коллекцию в Postman и выполняйте запросы. Пока проект не научится обрабатывать запросы — от коллекции не будет практической пользы.

Коллекция запросов — это вспомогательный инструмент для проверки API, применяйте его по собственному усмотрению или не применяйте вовсе, решение — за вами.

Успешное выполнение всех запросов из Postman-коллекции не означает, что задание выполнено: решающее слово остаётся за тестами.

Как работать с коллекцией?

Для каждого запроса в коллекции заготовлен пример ожидаемого ответа. Если вернувшийся ответ не будет соответствовать ожидаемому — Postman покажет сообщение об ошибке. Сообщения могут быть по-русски и по-английски, читайте их внимательно, они помогут обнаружить ошибку. Держите словарь под рукой, некоторые английские термины могут быть вам незнакомы.

Если вдруг вы обнаружите, что проверка API через коллекцию запросов в Postman в чём-то противоречит тестам на платформе — приоритет всегда за тестами, ориентируйтесь именно на них.

Прежде чем начать работу с коллекцией — подготовьте Django-проект:

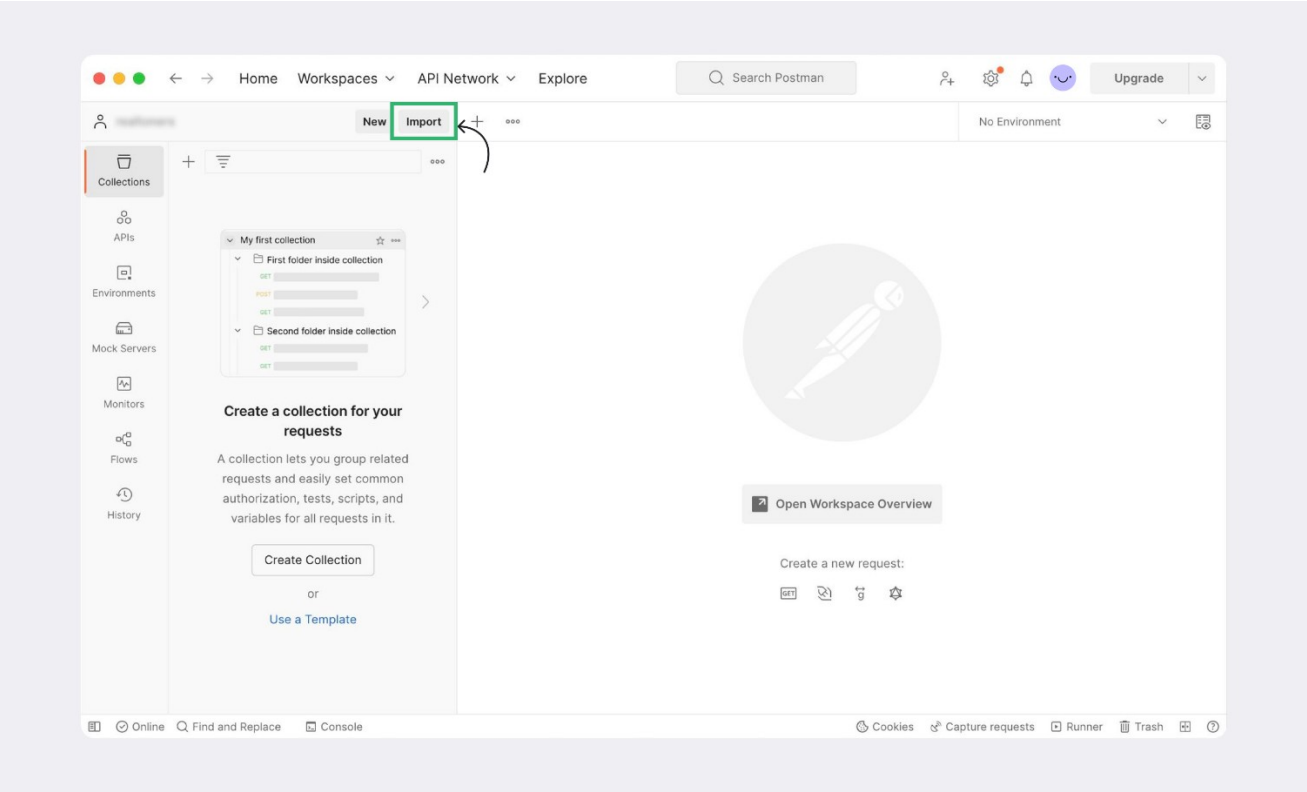
1. Проверьте, что виртуальное окружение развёрнуто и активировано, зависимости проекта установлены.
2. Перейдите в директорию *postman_collection* и командой `bash set_up_data.sh` запустите bash-скрипт для создания необходимых для работы коллекции объектов в базе данных.

Внимание, скрипт предварительно очищает существующую базу данных.

3. Перейдите в директорию с файлом *manage.py* и запустите веб-сервер разработки.

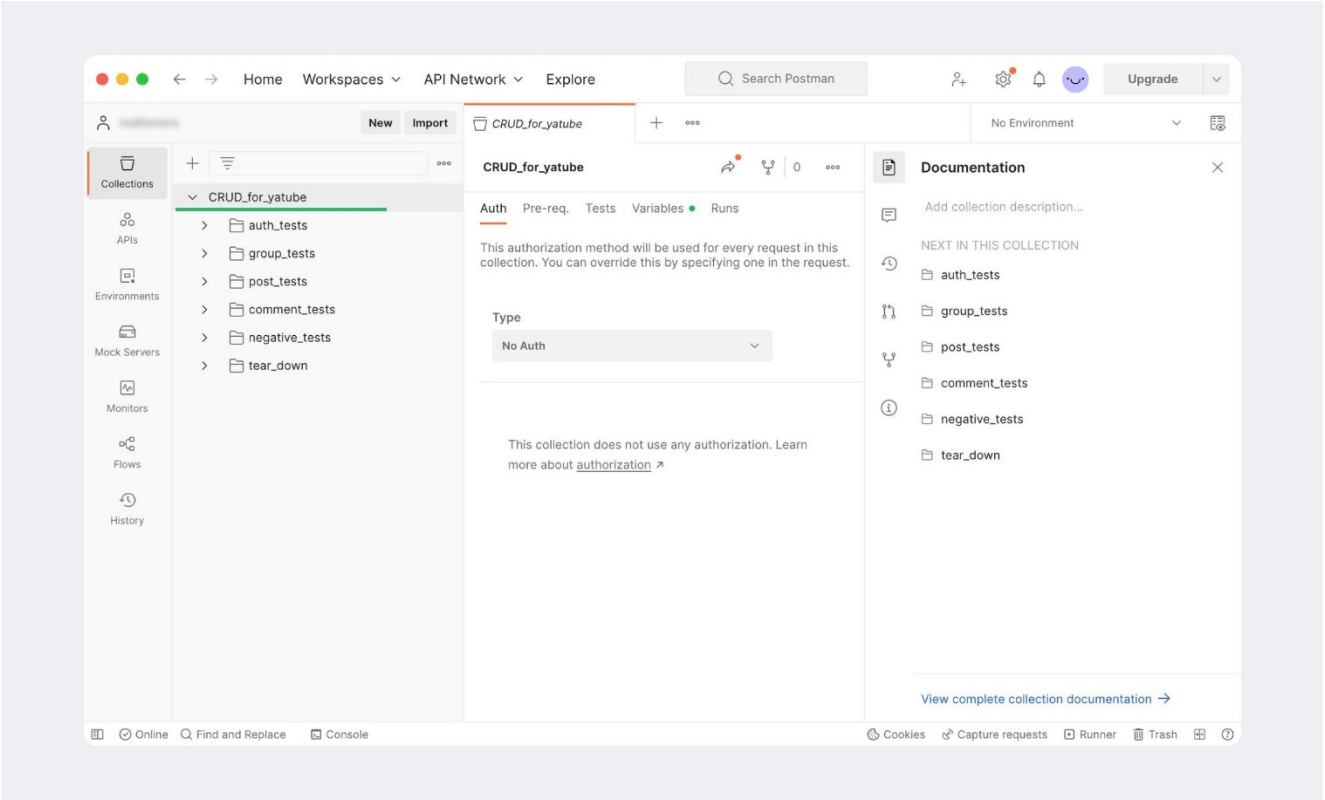
Особенности запуска и настройки коллекции описаны в файле README.md. Чтобы начать работу с коллекцией запросов — выполните следующие шаги:

1. **Импорт коллекции:** Откройте Postman и нажмите на кнопку "Import" в верхнем левом углу.

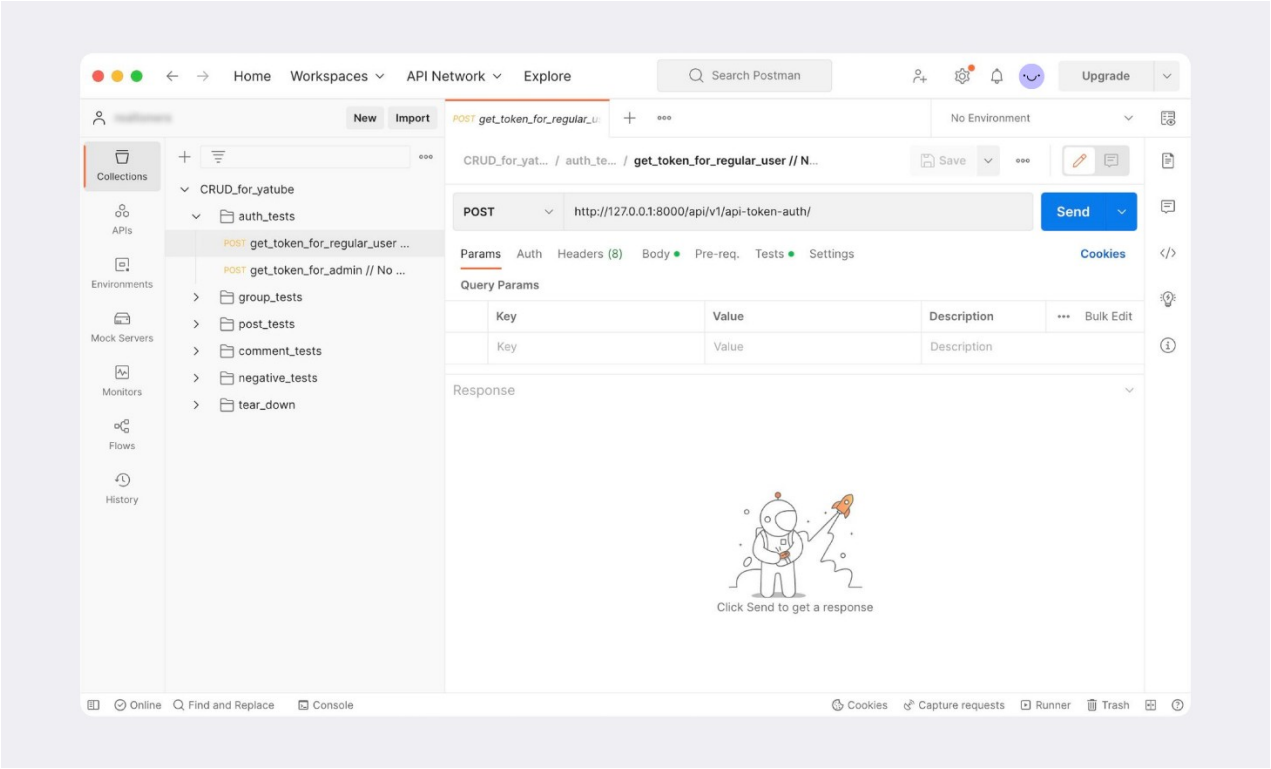


Во всплывающем окне будет предложено перетащить файл с коллекцией или выбрать файл через окно файлового менеджера. Выберите файл коллекции *CRUD_for_yatube.postman_collection.json*.

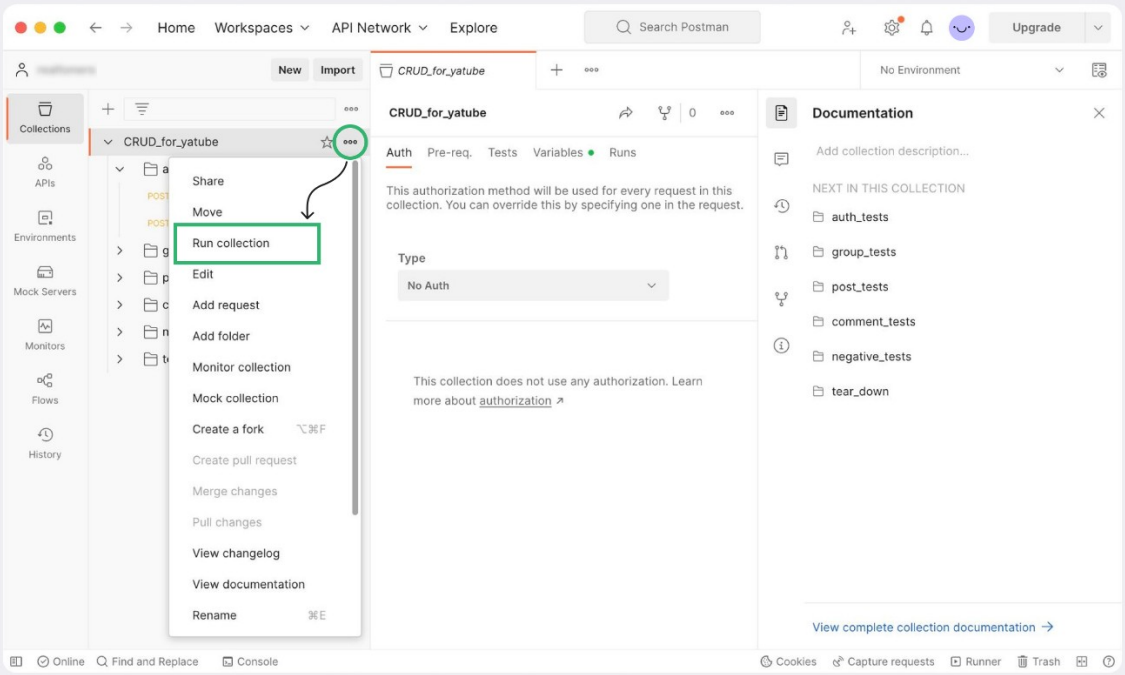
2. **Просмотр коллекции:** После импорта коллекция появится в левой панели. Нажмите на её название, чтобы увидеть все запросы.



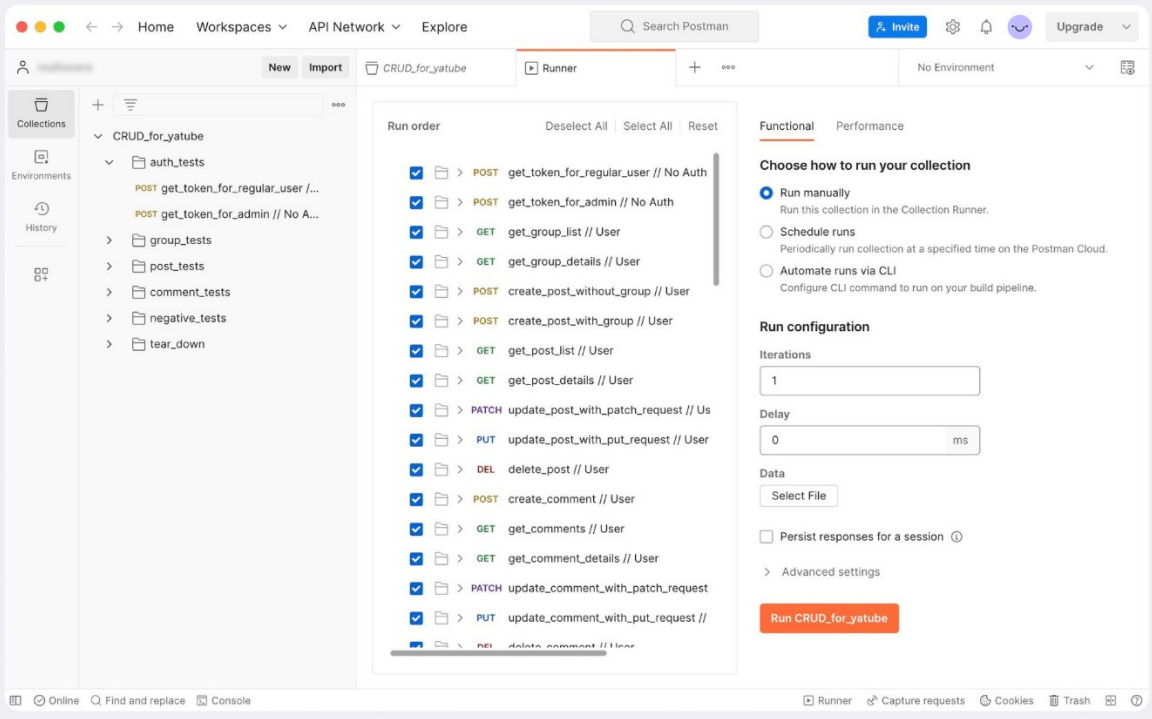
3. **Настройка и проверка запросов:**
   - Выберите запрос в коллекции, нажав на него.
   - В верхней части окна вы увидите тип HTTP-запроса и URL-адрес API. При необходимости эти данные можно изменить, например — добавить параметры запроса, заголовки или тело запроса в соответствующих вкладках под строкой URL.



4. **Сохранение изменений в запросах:** Если вы внесли изменения в запрос — сохраните его, нажав на кнопку "Save" в правом верхнем углу.
5. **Запуск всех запросов в коллекции:** Запросы можно отправлять по одному или все сразу. Для последовательного запуска всех запросов используйте функцию **Runner**. Для этого нажмите на три точки напротив названия коллекции и в выпадающем списке выберите *Run collection*.



В центре экрана появится список запросов из коллекции, а в правой части экрана — меню для настройки параметров запуска.



В правом меню включите опцию **Persist responses for a session**, это даст возможность посмотреть ответы API.

Для запуска коллекции нажмите кнопку `Run [имя вашей коллекции]`.

6. **Экспорт коллекции:** Если вы внесли изменения в коллекцию и хотите сохранить её для дальнейшей работы или для обмена с коллегами, вы можете её экспортировать. Для этого в выпадающем меню коллекции выберите **Export**.

Если не получается отправить проект на проверку смотрите шпаргалку.
Сдать работуВперёд

Чек-лист проекта «CRUD для Yatube»

Общие пояснения

После выполнения задания нужно проанализировать результат. Это называется **ревью**.

Если рядом нет опытного разработчика, который укажет на ошибки, даст советы и похвалит за удачные места, можно **проверить себя самостоятельно**. Для этого мы подготовили критерии, которые помогут оценить и улучшить сделанную работу.

Раздел «Технологии для освоения»

Убедитесь, что понимаете термины и освоили перечисленные технологии. Если всё-таки что-то забыли или не поняли, повторите уроки. Возможно, это придётся сделать несколько раз.

Раздел «Обязательные критерии»

Здесь мы собрали условия, без которых задание не считается выполненным.

- Важно: все автотесты должны быть пройдены успешно. Если автотесты найдут ошибки, вы увидите сообщение: оно укажет, на что обратить внимание, чтобы пройти тест. Учтите эту информацию и доработайте код.
- Некоторые пункты этого блока повторяют пункты техзадания, которые не попали под проверки автотестами.
- Другие пункты уточняют техническое задание.
- А остальные указывают на принципы разработки.

Раздел «Рекомендации»

Здесь мы рассказываем, как можно изменить код, чтобы улучшить проект. Важно: это не инструкции к выполнению, а именно советы. Вы сами можете выбрать, чему следовать.

Если хотите и будет время, потренируйтесь — это поможет перейти на новый уровень знаний. Но учитывайте, что любые попытки изменить код порождают новые ошибки. Поэтому, если что-то исправляете, сверяйте изменения с требованиями из раздела «Обязательные критерии».

Чек-лист

Технологии для освоения

- Структура DRF-проекта.
- Сериализаторы в и из JSON.
- Вьюсеты, роутеры для маршрутов.

Обязательные критерии

- [ ] Все автотесты пройдены успешно.

Ответить

Модели (models.py)

- [ ] Все модели представлены в админке.

Ответить

Сериализаторы (serializers.py)

- [ ] В сериализаторах для «Поста» и «Комментария» тип для поля «автор» нужно подобрать точно. Типы `CharField` или `SerializerMethodField` не годятся, так как не задействуют специализацию «Связь между таблицами».

Тип `StringRelatedField` не подходит, потому что правильно работает лишь случайно: опирается на преобразование в строку для объектов типа `User`. Поэтому нужен самый гибкий из штатных `**RelatedField` типов.

Недопустимо добавлять в сериализаторы поля, для которых описания в модели уже достаточно для работы DRF. Например, будут лишними все поля:

- у группы;
- у поста, кроме «автор»;
- у комментария, кроме «автор».

Ответить

URL, маршруты, роутеры (urls.py)

URL запроса выглядит так: `api/v1/что-то-ещё/`. Регистрация маршрутов находится в трёх местах: - в проекте, то есть в модуле `yatube_api/urls.py`, - в приложении, то есть в модуле `api/urls.py`, - в роутере, то есть внутри вызовов `register()`.
Что касается местоположения сегментов URL, в проекте должен остаться только `api/`, чтобы проект мог раздавать разрешения входящим запросам. Сегмент `v1/` не должен попадать в роутер: он общий для всех URL этого роутера.

Контроллеры для всех запросов, кроме получения токена, должны регистрироваться в роутере. А значит, они все должны быть вьюсетами.

При регистрации в роутере вьюсета для «Комментария» есть сложное место: нужно применить регулярное выражение для указания «только цифры». Правильное регулярное выражение — `\d+`.
Обратный слеш у регулярного выражения нельзя применять без дополнительных изменений: его нужно записать через `\\` или перед объявлением строки поставить префикс префикс `r`. То есть писать не `'posts/...'`, а `r'posts/...'`.
Этот префикс лучше всегда добавлять к строкам с регулярными выражениями. Даже если в них ещё не появился обратный слеш, скорее всего, он появится позже.

По техническому заданию для «Группы» нужны не все CRUD-запросы. Нельзя выполнять это требование через настройку маршрутов. Нужно сделать с помощью вьюсета.
Ответить

Контроллеры (views.py)

Все контроллеры должны быть вьюсетами.

По техническому заданию для «Группы» нужны не все CRUD-запросы. Нужно подобрать из DRF-классов вьюсет, подходящий для наследования.

В вьюсете «Комментария» два раза потребуется расчёт поста: для кверисета и для создания. Не повторяйтесь: нужно разместить этот расчёт в новом методе.

Расчёт поста в вьюсете «Комментария» должен быть надёжным. Не доверяйте параметрам контроллеров. Они приходят из ненадёжных URL, которые могут набрать руками или взять из устаревших закладок. Проконтролируйте, что пост с таким ключом есть в базе. Если нет, должна быть ошибка 404.
Ответить

Рекомендации

Все тесты в Postman пройдены успешны.
Ответить

Сериализаторы (serializers.py)

В сериализаторе «Комментария» нужно настроить поле «Пост» только на чтение. Создавать само поле в сериализаторе и указывать настройку через его параметр `read_only` избыточно. DRF сам создаст поле, данных из модели хватает.
Лучше задействовать подкласс `Meta` и его настройку для перечисления полей только на чтение.
Ответить

URL, маршруты, роутеры (urls.py)

Полезно вводить в код поддержку версий как можно раньше. Лучше дать роутеру имя, содержащее номер версии API. По техническому заданию это `v1`.

Зачем это нужно:

- Сервер будет развиваться.
- После обкатки текущего API-набора запросов появится необходимость сменить сам набор.
- Так нужно делать, чтобы старый набор (или все прежние наборы) продолжал свою работу.
- Для этого в URL уже есть версия. Переход на новую не отменит существования прежней. А вот код для поддержки прежней версии нужно изолировать. Этим и будут заниматься несколько роутеров — каждый с указанием своей версии.



Лучше всегда добавлять параметр `basename` к каждому вызову `register()`. У него есть прямое и косвенное действие:

- **Прямое:** для вычисления URL по этому имени.
- **Косвенное:** можно не писать в контроллер-классах поле queryset, если есть метод `get_queryset()`.

Ситуация подобна использованию параметра `name` при регистрации маршрутов через `path()`. Этот параметр необязательный, но в серьёзном проекте точно пригодится.

Ответить

Контроллеры (views.py)



Расчет кверисета для вьюсета «Комментария» лучше выполнить, опираясь на «Поле связи», то есть через значение параметра `related_name` из модели. Например, все комментарии для поста, лучше получать через `пост.comments.all()`, если в модели задано `related_name='comments'`.

Ответить

Как вам урок?

К следующему уроку